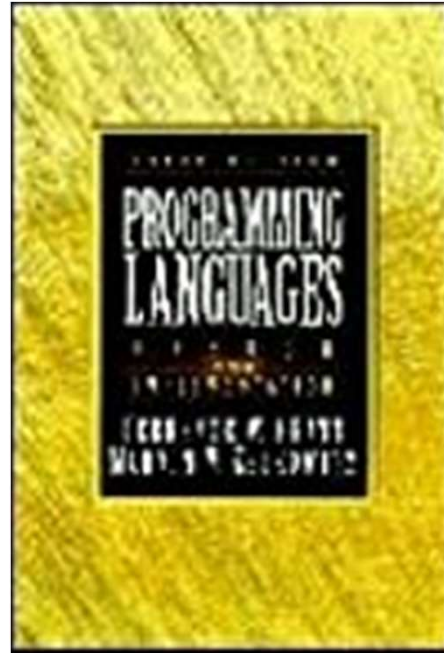




Controle de Subprograma

Interação entre Subprogramas

Parte 2 de 3

Dados e Ambiente de Referência **Locais**

32

- Ambiente local de um subprograma Q
 - ▣ Nomes dos seus parâmetros formais.
 - ▣ Nomes de suas variáveis declaradas.
 - ▣ Nomes dos subprogramas aninhados.
 - ▣ O identificador Q **não é parte** desse ambiente.

Dados e Ambiente de Referência **Locais**

Regras

33

- **Regra de escopo estática**

Referência a X no corpo de Q está **relacionado a declaração** de X em Q .

- **Regra de escopo dinâmica**

Referência a X , durante a execução de Q , refere-se a **associação de X na ativação corrente** de Q .

- São facilmente mantidas **consistentes**.

Dados e Ambiente de Referência **Locais**

Implementação

34

□ **Regra de escopo estática**

O compilador mantém uma **tabela dos identificadores locais**, enquanto compila o corpo de Q. Quando uma referência é feita a um identificador, ele **olha nessa tabela primeiro**.

Dados e Ambiente de Referência **Locais**

Implementação

35

- **Regra de escopo dinâmica**

A implementação em **ambiente de referência local** tem 2 enfoques:

- **retenção**

- **eliminação**

- Cada um desse enfoque atribui diferentes semânticas para referências locais.

Dados e Ambiente de Referência **Locais**

Implementação: Regra de Escopo Dinâmica

36

- **Retenção** (Cobol e diversas versões de Fortran)
 - ▣ As variáveis locais retidas **preservam seus valores** entre chamadas de um mesmo subprograma.
 - ▣ Variáveis retidas são armazenadas no **segmento de código.**
 - ▣ O **mesmo resultado** pode ser obtido com **variáveis não locais.**

Dados e Ambiente de Referência **Locais**

Implementação: Regra de Escopo Dinâmica

37

- **Eliminação** (C, Pascal, Ada, LISP, APL)
 - ▣ Os OD, declarados localmente a Q, **são criados a cada chamada** de Q.
 - ▣ O ambiente local a Q é **eliminado ao final da execução** daquela chamada de Q.
 - ▣ As variáveis não retidas e parâmetros de Q são incluídas no **segmento de dados da área de ativação** (registro de ativação) de Q.

Dados e Ambiente de Referência **Locais**

Retenção ou Eliminação?

38

```

program retencao(...).
  procedure P(...);
    RET var x,y: integer = 1;
    var t: integer;
    begin write(x); t:=x; x:=y; y:=t+y; end;
  procedure fibonacci (n:integer);
    begin
      while n>0 do begin n:=n-1; P; end;
    end;
  (* corpo do programa principal *)
  begin
    ...
    fibonacci(5);
    ....
  end.

```

□ Resultados

a) com **retenção**

n	P	x	y	t	x	y
5	-	-	-	-	-	-
4	1	1	1	1	1	2
3	2	1	2	1	2	3
2	3	2	3	2	3	5
1	4	3	5	3	5	8
0	5	5	8	5	8	13

b) com **eliminação**

x : 1 1 1 1 1

Dados e Ambiente de Referência **Locais**

Implementação:

39

- Tanto a abordagem de **retenção**, quanto a de **eliminação** são facilmente implementadas no modelo de “**registro de ativação**”. Contudo, alguns pontos merecem atenção:

Dados e Ambiente de Referência **Locais**

Retenção e Eliminação: Pontos Importantes

40

□ **Subprograma sem recursão**

Retenção e eliminação: **mesma implementação** pois somente existirá um registro de ativação, o qual pode ser alocado estaticamente. **Inicialização** de OD pode resultar em duas implementações diferentes:

- inicializa a **cada chamada** (atribuição no código do subprograma)
- inicializa só na **1ª. chamada** quando cria o segmento de código do subprograma

Dados e Ambiente de Referência **Locais**

Retenção e Eliminação: Pontos Importantes

41

- **Ambos** retenção (em **segmento de código**) e eliminação (via **registro de ativação**)

PL/1 e **C**

- ▣ variáveis estáticas → retidas

- ▣ variáveis automáticas (default) → eliminadas

ALGOL 60

- ▣ variáveis OWN → retidas

Algumas implementações de COBOL e Fortran apresentam enfoque de retenção.

Dados e Ambiente de Referência **Locais**

Retenção e Eliminação: Pontos Importantes

42

- **Nomes de subprogramas são sempre retidos**

A associação do nome de um subprograma declarado no ambiente local e sua definição pode ser representada por um **ponteiro para a área do segmento de código** representando o subprograma.

- **Parâmetros formais**

São inicializados a cada chamada, logo **não podem ser retidos** entre chamadas.

Dados e Ambiente de Referência **Locais**

Retenção e Eliminação: Pontos Importantes

43

□ **Recursão**

- **Múltiplas ativações** geram múltiplos registros de ativações armazenados simultâneos, no **pilha central**.
 - Exemplos da variável “x” podem existir em registros de ativações distintos, sendo o enfoque de **eliminação desejável**.
 - Contudo pode ser **útil haver variáveis retidas**, acessíveis às **várias ativações** do mesmo subprograma, mantendo os valores entre as chamadas.

Compartilhamento de Dados em Subprogramas

44

- Dados podem ser compartilhados através de:
 - **transmissão de parâmetros** ou
 - acesso a dados em **ambientes não locais**, segundo os enfoques de:
 - ambiente comum explícito e
 - ambiente não local implícito, baseado em:
 - escopo dinâmico,
 - escopo estático,
 - herança (cap. 8 do Pratt, o qual não veremos)

Parâmetros e Transmissão de Parâmetros

45

- O **principal método** para compartilhamento de dados entre subprogramas é a **transmissão** explícita de **parâmetros** e **resultados**.
- Com ambiente **não local**, o compartilhamento é obtido tornando alguns nomes **visíveis** aos subprogramas que os utilizam.
- Na transmissão de parâmetros e resultados, OD podem ser transmitidos ou recebidos **sem** nomes associados.

Transmissão de Parâmetros

46

Compartilhamento sem vinculação de nomes:

`x = mdc (x1,x2);`

- ▣ x recebe um valor retornado

`y = mdc (12,15);`

- ▣ 12 e 15 não são nomeados

`x1 = sqrt(y);`

- ▣ x1 recebe um valor retornado

`x2 = sqrt(100);`

- ▣ 100 não é nomeado

Transmissão de Parâmetros

Parâmetros Atuais e Formais

47

- Argumento:
 - ▣ nome do OD transmitido como parâmetro ou operando.
- Resultado:
 - ▣ refere-se ao OD retornado ao final de um subprograma.
- Os argumentos (ou resultados) de um subprograma podem ser obtidos (ou retornados) via:
 - passagem de parâmetros,
 - referência não local,
 - arquivo
 - registradores

O resultado também pode ser retornado via o nome de procedimento do tipo função.

Transmissão de Parâmetros

Parâmetros Atuais e Formais

48

- **Parâmetro Formal**

- ▣ É um tipo particular de OD local ao subprograma
- ▣ É nomeado e declarado na especificação do subprograma.

Exemplo:

```
void Sub (int x; char y);
```

Nomeia os objetos x e y e declara seus tipos.

- **Parâmetro Atual**

- ▣ É o OD compartilhado pelo **chamador** do subprograma. Pode ser:
 - OD local ao chamador,
 - parâmetro formal do chamador,
 - OD não local mas visível ao chamador,
 - resultado de expressão **diretamente transmitida**.

Transmissão de Parâmetros

Estabelecendo Correspondência

49

Posicional:

- Parâmetros formais e atuais são relacionados com base na **posição relativa nas suas listas respectivas**, considerando a mesma ordem em que aparecem e seus tipos.

Explicitada por Nome:

- Em algumas LP, a correspondência pode ser estabelecida pelos nomes dos parâmetros, independente da ordem, para tipos idênticos.

Exemplo (Ada):

Sub (Y $\Rightarrow$ B, X $\Rightarrow$ 27);

Transmissão de Parâmetros

Estabelecendo Correspondência

50

□ Coincidência da Quantidade de Parâmetros

- ▣ Algumas linguagens **não exigem** que a quantidade de parâmetros **coincida** (LISP, HASKEL, GOFER e HUGS):

```
a) (defun inversao(Xs Ys)      b) > let mais5 = (5+)
    (cond                      in map mais5 [1, 2, 3]
      ((null Xs) Ys)          > [6, 7, 8]
      (T (inversao (cdr Xs)
                    (cons (car Xs) Ys))))
    ))
INVERSAO
$ (inversao '(a b c d))
(D C B A)
```

T – representa true (executado se Xs não vazia), CAR - cabeça da lista, CDR - cauda da lista

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

51

□ *Call by Name (Algol)*

- ▣ A expressão formando o parâmetro **atual** é substituída onde o parâmetro **formal** aparece, **SEM** nenhuma **avaliação**, antes da execução do subprograma. Requer indicar **o ambiente de referência** a ser usada **para avaliar essa expressão**.
- ▣ O argumento é **avaliado** em cada ponto **onde é referenciado**.
- ▣ Implementação:
 - vista como chamada a subprograma sem parâmetro,
 - resulta na avaliação do parâmetro atual (no ambiente de referência do chamador)
 - retorna valor como um resultado passado para o subprograma

Implementação é cara, por isso esse método não é popular.

Transmissão de Parâmetros

Exemplo de *Call by Name*

52

```
begin
  integer I, N; ...
  real procedure intervalos (integer A,B);
  begin ....
    while A<100 do
      begin
        write(<“[,5,“,“,5,“]”,/ >,
          A+3,B); I:=I+10;
      end;
    end;
  ....
  I:=1;
  intervalos (I,I+10);
end.
```

Resultados:

[4,11],	% I=1
[14,21],	% I=11
[24,31],	% I=21
[34,41],	% I=31
[44,51],	% I=41
[54,61],	% I=51
....	
[94,101],	% I=91

"A" pode ser visto como um alias para "I".

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

53

□ **Call by Reference (é método mais comum)**

- ▣ O parâmetro **formal** é um **ponteiro** (*l-value*) para o OD associado ao parâmetro **atual**. Desta forma é criado um **alias** para o OD atual.
- ▣ O parâmetro **atual** é avaliado, gerando um **endereço** (*l-value*) do OD associado (aquele sendo passado como parâmetro). O OD não muda de lugar.
- ▣ Durante a execução do subprograma chamado, cada **mudança** no parâmetro formal, **altera** o OD passado como parâmetro real (atual) no ambiente do chamador.

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

54

□ *Call by Value* (C, java, C++)

- ▣ Semelhante a passagem por referência, mas ao invés do *l-value* do OD associado ao parâmetro atual, **passa-se o *r-value***.
- ▣ Os parâmetros **formais** são **iniciados com valores dos *r-values*** dos parâmetros **atuais**.
- ▣ Alterações em parâmetros formais **não afetam** os parâmetros atuais. Elas são perdidas com o término do subprograma.
- ▣ Parâmetros **formais** são **variáveis locais** ao subprograma chamado, do mesmo tipo dos parâmetros atuais.
- ▣ *Call by value* é chamado ***by copy*** na linguagem **Ada 95**.

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

55

□ **Call by Value-Result (Algol W)**

- O parâmetro **formal** é implementado como uma **variável local** do mesmo **tipo** do parâmetro **atual** correspondente.
- O **valor** do parâmetro **atual** é utilizado para **iniciar o r-value** do parâmetro **formal** correspondente.
- Durante a **execução** do subprograma, **alterações** em parâmetros formais **não afetam** os parâmetros atuais.
- Ao **término** do subprograma, os **r-values** dos parâmetros formais são **copiados** para os parâmetros **atuais**.
- O parâmetro **atual mantém** seu **valor até o término** do subprograma chamado, sendo alterado no término da execução
- **Call by value-result** foi proposto por Nicklaus Wirth no projeto da linguagem Algol-W.

Algol-W é um sucessor do Algol, anterior ao desenvolvimento do Pascal.

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

56

□ *Call by Constant Value*

- ▣ O **valor** do parâmetro **formal**, passado como constante, **não pode ser alterado** durante a execução do subprograma.
- ▣ Esse parâmetro **não pode ser passado** (como parâmetro atual) para outro subprograma, **exceto se por *call by constant value***.
- ▣ Durante a execução do subprograma, os parâmetros formais podem ser vistos como constantes locais.
- ▣ Implementado como ***“by value”*** ou como ***“by reference”***.
- ▣ *Call by value* e *call by constant value* **impedem alterações** nos valores dos parâmetros **atuais**, **tornando-os argumentos de entrada** para o procedimento chamado.

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

57

□ *Call by Result*

- ▣ O parâmetro formal transmitido *by result* é **utilizado** apenas **para retornar** um resultado para o procedimento chamador.
- ▣ O **valor inicial** do parâmetro atual **não pode ser utilizado** no procedimento chamado.
- ▣ O parâmetro **formal** é uma **variável local** sem inicialização ou com a inicialização própria para as variáveis locais implementada pelo projetista da LP.
- ▣ Quando o procedimento chamado **termina**, o **valor** do parâmetro **formal** é **copiado** para o parâmetro **atual** associado.

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

58

□ Implementações utilizadas em algumas LP

□ A maioria implementa um ou dois dos métodos vistos.

■ **Fortran:** passagem por referência.

■ **Pascal:** passagem por referência (usando o sufixo var) e por valor (default).

■ **C:** passagem por valor. O uso de ponteiro permite ao programador utilizar passagem por referência:
Exemplo:

```
void sub (int * x); ....      /* declaração da função */  
sub (&i);                    /* &i é o l-value de i */
```

■ **Algol-W:** passagem por valor-resultado.

Transmissão de Parâmetros

Métodos para Transmissão de Parâmetros

59

□ Semântica da Transmissão

- O **programador** deve **saber quais os métodos** para transmissão de parâmetros **foram implementados** na LP para depois **decidir** qual deles **vai usar**.
- Ada adota uma abordagem diferente: informa o significado pretendido ao invés do método de transmissão implementado.
Exemplo: `procedure sub (X: in integer, Y: out float, Z: in out float);`
- TD elementares são passados por “valor constante” (*in*) ou por “valor-resultado” (parâmetro do tipo *out* ou *in out*).
- TD estruturados (arrays e registros) são passados por “referência”.

Transmissão de Parâmetros

Passagem de Subprograma como Parâmetro

60

- O uso do nome de um **subprograma** como **parâmetro atual** na chamada de um subprograma **traz dificuldades**:
 - ▣ Como **checar os tipos dos parâmetros** do subprograma, durante a tradução, se seu **nome só será conhecido** ao ser passado como um parâmetro **atual**?
 - Solução:
 - A **assinatura** do subprograma **precisa ser especificada** como parâmetro formal.
 - Inclui os **tipos dos parâmetros** do subprograma

Exemplo:

```
procedure Q (x:integer; function R(y,x: integer): integer);
```

Transmissão de Parâmetros

Passagem de Subprograma como Parâmetro

61

- O uso do nome de um **subprograma** como **parâmetro atual** na chamada de um subprograma **traz dificuldades**:
 - ▣ Se há **referências** a variáveis **não locais** no **subprograma** qual **ambiente usar** para **resolve-las**?
 - E se o chamador possuir identificadores com o **mesmo nome** que as referências não locais, qual usar?
 - Solução:
 - a referência **não local** significa o mesmo se o **subprograma fosse chamado** no **ponto** em que aparece como um **parâmetro atual na lista de parâmetros**.
 - **Passa-se** de fato o **par** **<cp,scp>**, o que define o ambiente de referência.

CP: ponteiro para segmento do código do subprograma chamado. SCP: ponteiro de encadeamento estático (aponta para o registro de ativação do subprograma local)

Transmissão de Parâmetros

Passagem de Subprograma como Parâmetro

62

- Exemplo em Pascal (subrotina aninhada)

```
program main; var X: integer;
  procedure Q (var l:integer; function R (J:integer):integer);
    var X:integer; begin X:=4;
    write ("Em Q, antes de chamar R, l=", l, "X=", X); l:= R(l);
    write ("Em Q, depois de chamar R, l=", l, "X=", X);
  end;
  procedure P; var l:integer;
    function FN (K:integer):integer; begin X:= X+K; FN:= l+K;
    write ("Em P, l=", l, "K=", K, "X=", X); end;
    begin l:=2; Q(X,FN); write ("Em P, l=", l, "X=", X);
  end;
  begin X:=7; P; write ("Em Main, X=", X);
end.
```

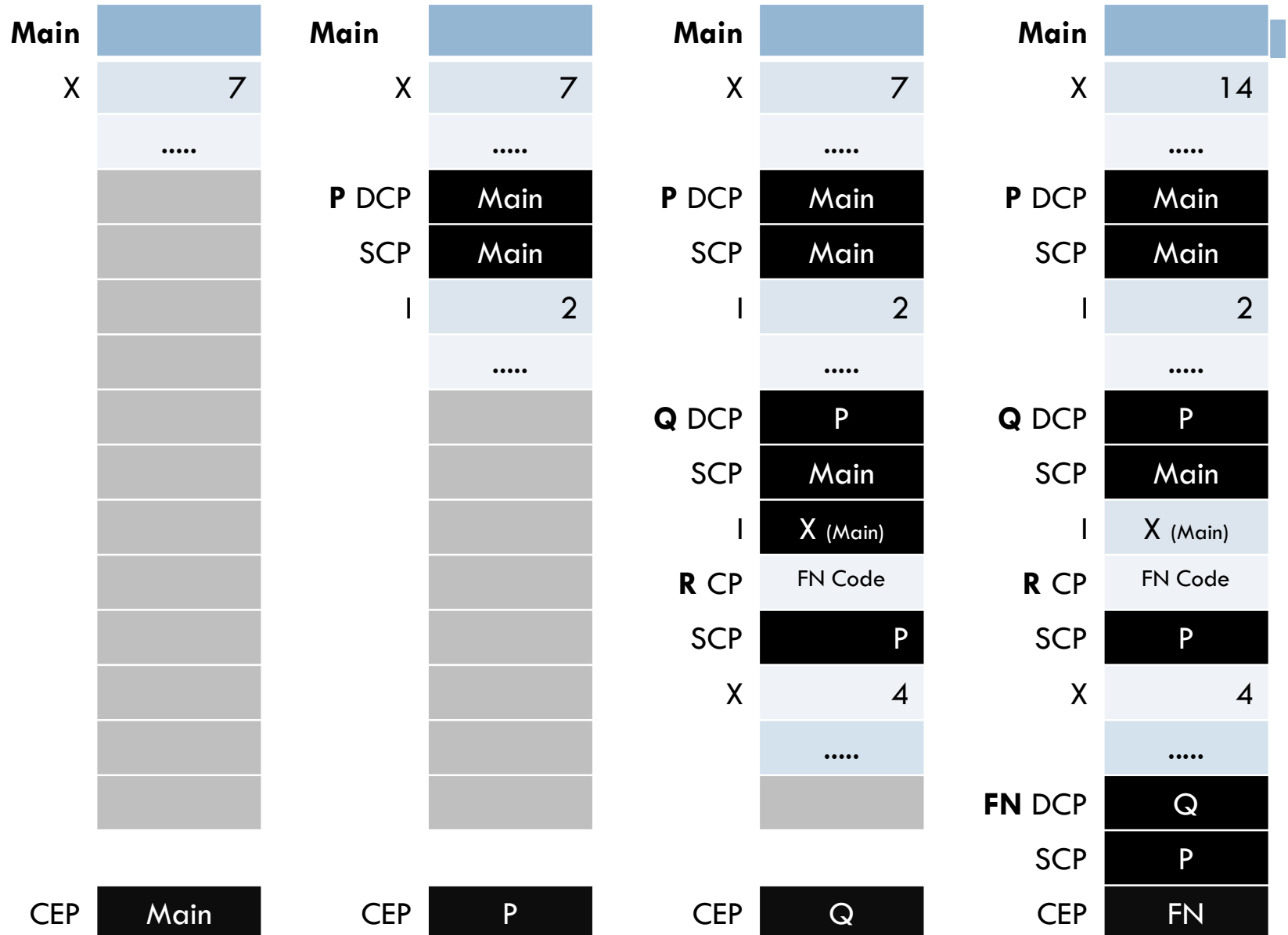
Em FN, l e X são variáveis não locais

Passagem de Subprograma como Parâmetro

Exemplo em Pascal: Pilha de Execução

63

Esse slide será melhor compreendido após
apresentação de escopo estático e escopo dinâmico



Transmissão de Parâmetros

Rótulo de Comando Passado como Parâmetro

64

- Passar um rótulo para ser usado em **goto** no subprograma **chamado**. Esse mecanismo traz **dificuldades**:
- Para qual ativação do segmento de código que contém o rótulo se deve transferir o controle?
 - ▣ O **goto** **não pode simplesmente alterar o CIP** pois **várias ativações** podem estar usando o **mesmo segmento de código**.
 - ▣ O rótulo deve ser passado com a **indicação de qual registro de ativação usar**.
 - ▣ É **necessário** passar o par **(ip, ep)** como parâmetro:
 - ip é o endereço de instrução (em um segmento de código)
 - ep é o endereço do ambiente (registro de ativação) a que se refere.

Transmissão de Parâmetros

Rótulo de Comando Passado como Parâmetro

65

- Passar um rótulo para ser usado em **goto** no subprograma **chamado**. Esse mecanismo traz **dificuldades**:
- Como implementar o desvio para um parâmetro rótulo?
 - ▣ Não é suficiente desviar o controle para (ip, ep). **Requer finalizar** o subprograma em execução e **tratar a cadeia dinâmica de ativações** de subprogramas.
 - ▣ A implementação usual é:
 - finalizar a execução do subprograma ativo.
 - finalizar a execução de todos os seus chamadores até encontrar um que corresponda a ativação designada no **goto** (isto é, se refira ao **ep** do par (ip, **ep**)).
 - os valores finais dos parâmetros passados como **resultado** ou **valor-resultado** dependem da implementação da LP.